Tractive System Status Indicator w/ two boards & firmware

The Tractive System Status Indicator (TSSI) is a safety feedback system required by FSAE rule EV.5.11.5. It indicates high voltage safety concerns. It monitors the Battery Management System (BMS) and Insulation Monitoring Device (IMD). FSAE rules require the safety indicator to be w/o programmable logic, so you have to use electrical components. But for the sake of the project/firmware we will use some programmable components.

- PCB 1 (Power & Safety Board): Implements 12V power regulation, signal conditioning for inputs, non-programmable fault logic, discrete hardware fault-latching, and an inter-board header.
- PCB 2 (Display & Communication Board): Plugs directly into PCB 1. Implements solid Red and Green indicator LED drivers, 3.3V power regulation, MCU core hardware w/ input level-shifting, and a CAN transceiver interface.

Inputs

- 12V Battery: A single +12V DC input that powers PCB 1 and supplies power through to PCB 2.
- BMS_FAULT (12V Digital Signal): A signal from the Battery Management System.
    - HIGH (> 10V): Indicates that the BMS has not detected a fault (system operating w/o active BMS faults).
    - LOW (< 0.5V): Indicates that a BMS fault is active.
- IMD_FAULT (12V Digital Signal): A signal from the Insulation Monitoring Device.
    - HIGH (> 10V): Indicates that the IMD has not detected a fault (system operating w/o active IMD faults).
    - LOW (< 0.5V): Indicates that an IMD fault is active.

Outputs

- Green Indicator LED State (PCB 2): The electrical state of the solid green indicator LED.
    - ON: When no faults have been detected (operating w/o safety issues).
    - OFF: When a fault has occurred.
- Red Indicator LED State (PCB 2): The electrical state of the solid red indicator LED.
    - ON: When a fault condition has been triggered.
    - OFF: When no faults have been detected.
- CAN Bus State (PCB 2): The transmission of diagnostic data over the vehicle network managed by the microcontroller and transceiver.

Requirements

- Initial state: When your circuit is powered on, evaluate BMS_FAULT and IMD_FAULT. If both are HIGH:
    - Green LED is ON, solid light.
    - Red LED is OFF.
- Fault state: When either the BMS_FAULT or the IMD_FAULT input (or both) drops to less than 0.5V at any time, the following must happen:

    - Green LED needs to turn OFF.
    - Red indicator LED needs to turn ON solid.
    - The circuit on PCB 1 must latch this fault state using electrical components (no code). The Red LED on PCB 2 cannot turn off until the 12V input is disconnected and reconnected. basically, ur circuit needs to remember if a fault has occurred while it has been powered on. When it loses power, it needs to lose any fault memories.
- Reset state: The latched fault memory can only be cleared by a power cycle, which is initiated by disconnecting and reconnecting the 12V input. After the reset, the circuit's behavior will revert to the Initial State.
- Software Monitoring: The microcontroller on PCB 2 reads signal inputs tapped from PCB 1 and handles communication over the vehicle network w/ zero reliance on software for the physical safety lights.

Power & Safety Board (Hardware #1)

- Scope: Design power supply regulation, input signals, and hardwired safety logic.
- Components to Include:
    - 12V Power Protection & 5V Regulator: TVS diode and reverse-polarity protection (ideally w/ a P-channel MOSFET w/ low voltage drop), and a 5V regulator to pass power to PCB 2.
    - Input Signal Conditioning: Resistor dividers and RC filters to step down and clean incoming 12V fault signals w/o introducing excessive signal delay.
    - Logic & Hardware Latch: Discrete logic gates (NOR/NAND) and a latch or flip-flop to lock in fault states until power is disconnected.
    - Inter-Board Header: Pin header routing 5V, GND, and signal taps over to PCB 2.

Display & Communication Board (Hardware #2)

- Scope: Design LED drivers, microcontroller system, input protection, and CAN hardware.
- Components to Include:
    - Indicator LEDs & Drivers: Green and Red LEDs driven by N-Channel MOSFETs using logic signals passed from PCB 1 (designed w/ proper current limiting).
    - 3.3V Regulator & MCU Subsystem: 3.3V LDO regulator, MCU chip footprint, decoupling caps, reset pull-up, and programming header.
    - GPIO Protection & CAN Transceiver: Resistor dividers for MCU, and a CAN transceiver IC w/ a standard FSAE 4-pin connector.

Firmware

- Scope: Write the microcontroller code for PCB 2.
- Key Tasks:
    - Design your own CAN 2.0A ID protocol to allow other devices to parse the information you send out or to know how to send your device (PCB 2) messages
    - With the designer for PCB 2, design your hardware such that you can configure your microcontroller to have external interrupts on the input pins to catch state changes (a fault

trigger), timestamp it, and send it on the CAN line as diagnostic data. Note that your interrupt service should be as **simple and as short as possible.** Do not do all of this logic inside a callback.

- Send a CAN message on the line at 1 - 10 Hz that contains fault data, error data, and an incremented counter (i.e. varies from 0-8 per message and is different on each send) to act as a "heartbeat" check
- WDT to automatically reboot your MCU upon hanging for a certain period of time